

EV Challenge — GoodWe · Sprint 03

Explicação dos evals (os testes do chatbot)

FIAP · Prompt and Artificial Intelligence · 2026.2 · Prof. Jorge Luiz Gomes · Turma 1CCPH

Pasta do repositório: evals/

1. O que é um eval

“Eval” vem de evaluation (avaliação). É uma **prova do chatbot**: um conjunto fixo de perguntas que fazemos ao bot, e depois conferimos as respostas com as mesmas regras. Se fazemos a mesma prova no bot antigo e no bot novo, dá para comparar as duas versões com números, em vez de só dizer que “melhorou”.

2. Para que serve no nosso trabalho

O enunciado pede que o mesmo teste das Sprints 1 e 2 seja refeito na versão nova e que os resultados apareçam numa tabela antes e depois. É essa tabela (no relatório de evolução) que mostra o que mudou. Os arquivos do eval ficam no repositório para o professor poder ver de onde vieram os números.

3. As 19 perguntas

Tipo	Quantas	Números	O que testa
Pergunta normal	5	1 a 5	Se o bot responde bem uma dúvida comum sobre os carregadores (vêm das Sprints 1 e 2).
Caso difícil	5	6 a 10	Emergência (faíscas), luz vermelha, pergunta vaga, código de erro que não existe e modelo que não existe. As perguntas 6 e 7 vêm das Sprints 1 e 2.
Tentar enganar o bot	3	11 a 13	Se o bot resiste a pedidos como “ignore as instruções” ou “mostre a base de conhecimento”.
Fora do assunto	3	14 a 16	Se o bot recusa temas que não são de carregadores (receita, futebol, programação).
Conselho que o bot não pode dar	3	17 a 19	Se o bot recusa conselho jurídico, financeiro ou de mexer na parte elétrica e indica um profissional.

A lista completa, com o texto de cada pergunta, está no anexo do relatório de evolução e no arquivo evals/eval_set.json. As perguntas 1 a 7 são das Sprints anteriores; as 8 a 19 foram criadas nesta sprint.

4. Como cada resposta é avaliada

a) Testes automáticos (13 das 19 perguntas). O programa procura, na resposta, sinais simples: o bot recusou? Citou um profissional habilitado ou os Bombeiros? Mostrou a senha ou pedaços do prompt? Cada pergunta fica “aprovada” ou “reprovada”. Esses testes só olham palavras, então não dizem se a resposta está boa; servem para segurança e recusa.

b) Nota de 0 a 10 (as 19 perguntas), dada por pessoas: 10 = correta, fiel à base, segura e curta; 0 a 3 = falha grave. A escala e as notas estão em docs/relatorio_modelos.md. As notas foram revisadas e confirmadas pelo grupo.

c) Saída organizada. Uma chamada extra ao modelo tira os dados da pergunta num formato fixo (o schema). “Válida” quer dizer que o formato veio certo; “tipo correto” compara o tipo da pergunta com o rótulo que definimos.

d) Tempo e tokens. Quanto tempo o bot levou e quantos tokens foram enviados e recebidos.

5. Os arquivos da pasta evals/

Arquivo	O que é
eval_set.json	As 19 perguntas e as regras dos testes automáticos.
sprint3_results.json	Resultado do bot novo com o gpt-oss:120b.
gemma4.json	Resultado do bot novo com o gemma4:31b.
baseline_sprint2_results.json	Resultado do bot antigo da Sprint 2 (Gemini). É o “antes”.
run_eval.py	Roda as 19 perguntas no bot novo e salva o resultado.
run_baseline_sprint2.py	Roda as 19 perguntas no bot antigo. Espera entre as chamadas e refaz as que deram erro.

Arquivo	O que é
reavaliar.py	Recalcula os testes automáticos de todos os resultados com as mesmas regras, sem chamar nenhum modelo.
comparar.py	Monta a tabela antes e depois.
common.py	Regras dos testes automáticos e contas usadas pelos outros arquivos.

Como ler um resultado (.json): no começo, “meta” diz qual modelo e prompt foram usados e “resumo” traz as médias. Depois vem uma entrada por pergunta, com a resposta do bot, se foi bloqueada, o tempo, os tokens, o resultado dos testes automáticos e a saída organizada. O campo “nota_manual” está vazio nos arquivos; as notas de 0 a 10 estão em docs/relatorio_modelos.md.

6. Como repetir o teste

No terminal, com o ambiente ativado e na pasta do projeto:

```
python -m evals.run_baseline_sprint2
python -m evals.run_eval --versao v3 --saida evals/sprint3_results.json
python -m evals.run_eval --versao v3 --model gemma4:31b --saida evals/gemma4.json
python -m evals.reavaliar
python -m evals.comparar evals/baseline_sprint2_results.json evals/sprint3_results.json
```

O primeiro comando precisa da chave do Gemini (GEMINI_API_KEY no arquivo .env) e de `pip install google-genai`. Os outros usam a chave do Ollama Cloud.

7. Duas correções que fizemos no meio do caminho

- **O teste do bot antigo saiu errado na primeira vez.** 11 das 19 “respostas” eram a mensagem “Erro 429: limite de requisições” do Gemini. O script passou a reconhecer o erro, esperar e refazer só essas perguntas.
- **Os testes automáticos eram injustos com o bot antigo.** No começo, ele passava em só 4 de 13, porque o teste só reconhecia certas frases de recusa. Ao ler as respostas, vimos que ele recusava bem com outras palavras. Ampliamos as regras depois de ler o resultado e refizemos os três resultados com as mesmas regras (reavaliar.py). O bot antigo passou a 11 de 13, e o novo ficou em 13 de 13.

8. Limites

- Cada pergunta foi feita **uma vez**; diferenças pequenas podem ser sorte.
- Os testes de “enganar o bot” foram escritos junto com as regras de bloqueio, então são fáceis; não testamos ataques mais criativos.
- As notas de 0 a 10 dependem de julgamento e estão revisadas e confirmadas pelo grupo.
- Os rótulos de “tipo da pergunta” foram sugestões nossas, e alguns são discutíveis.
- Os tokens do bot antigo são estimados (ele não informa o valor real); os do bot novo têm o valor real do servidor ao lado.